Proposal: Proton Integration with Playwright Framework

Prepared by: Altimetrik
Date: June 11, 2026
Version: 1.0

Table of Contents

---

1. Framework Background

Playwright Test Automation Framework

Playwright is a modern, open-source end-to-end test automation framework developed by Microsoft. It supports multiple browsers (Chromium, Firefox, WebKit) and provides robust tooling for UI and API testing. The existing framework has been designed and implemented to support automated regression and functional testing with the following characteristics:

- **Test Execution:** Automated test suites executed via CI/CD pipelines
- **Reporting:** JSON-based results (`results.json`) and PDF report generation
- **Test Management Integration:** Currently integrated with **Xray** (Jira-native test management), including pre-execution approval checks via API
- **Result Submission:** Automated test result upload to the test management platform post-execution

Proton Test Management Platform

Proton is the target test management platform to which the Playwright framework will be integrated. The integration will mirror the existing Xray integration pattern, leveraging Proton's REST API to:

- Create and manage test runs
- Submit test execution results
- Upload file attachments (PDF reports)
- Validate test case approval status prior to execution

This integration enables seamless traceability between automated test execution and Proton's test management workflows, replacing or supplementing the current Xray integration.

---

2. Project Scope

In Scope

The following activities are included in this engagement:

#	Activity	Description
1	**API Discovery & Analysis**	Analyse Proton's REST API documentation, endpoints, authentication mechanism, and data models relevant to test run creation, result submission, and attachment upload
2	**Framework Design**	Design the integration layer within the existing Playwright framework to support Proton as a test management target
3	**Development**	Implement the Proton integration module, including: test run creation, result mapping (Jira/Proton IDs), approval status validation, result submission, and PDF attachment upload
4	**CI/CD Pipeline Update**	Update existing pipeline configuration to support the Proton integration flow (approval check → execution → result upload)
5	**Testing & Validation**	End-to-end validation of the integration against a Proton sandbox environment, covering positive, negative, and edge-case scenarios
6	**UAT Support & Rework**	Support Proton stakeholders during UAT, address feedback, and deliver a final production-ready integration

Out of Scope

- Modifications to Proton platform itself
- Creation or management of Proton test cases
- Changes to existing Playwright test scripts beyond integration wiring
- Infrastructure provisioning for CI/CD pipelines
- Migration of historical test results into Proton

---

3. Architecture

High-Level Integration Architecture

```
┌───────────────────────────────────────────────────────────────┐
│                        CI/CD Pipeline                          │
│                                                               │
│  ┌──────────────┐    ┌──────────────────┐    ┌────────────┐  │
│  │  Trigger /   │───►│  Approval Check  │───►│  Execute   │  │
│  │  Schedule    │    │  (Proton API)    │    │  Tests     │  │
│  └──────────────┘    └──────────────────┘    └────────────┘  │
│                                                    │          │
│                                              ┌─────▼────────┐ │
│                                              │  Generate     │  │
│                                              │  results.json │ │
│                                              │  + PDF Report │ │
│                                              └──────────────┘ │
│                                                    │          │
│                                              ┌─────▼────────┐ │
│                                              │  Proton       │  │
```

```
│                                              │  Integration    │  │
│                                              │  Module         │  │
│                                              └─────────────────┘  │
└──────────────────────────────────────────────────────┼────────────┘
                                                       │
                                                       │
                                  ┌────────────────────▼───────────────┐
                                  │         Proton REST API            │
                                  │  ┌───────────────────────────────┐ │
                                  │  │ POST /test-runs               │ │
                                  │  │ POST /test-runs/{id}/results  │ │
                                  │  │ POST /test-runs/{id}/attach   │ │
                                  │  │ GET  /test-cases/{id}/status  │ │
                                  │  └───────────────────────────────┘ │
                                  └────────────────────────────────────┘
```

Integration Module Components

Component / File	Responsibility
`protonIntegration.groovy`	Core integration script — handles HTTP communication with Proton REST API, test run creation, result submission, PDF attachment upload, and error handling for both Standalone and Regression execution modes
`validateProtonTestCase.groovy`	Pre-execution approval validation — queries Proton API for test case approval status and gates pipeline execution (mirrors existing Xray approval check pattern)
`Jenkinsfile` (updated)	Pipeline definition updated to wire all three language runners (TypeScript, JavaScript, Python) through the Proton integration and approval validation steps

Technology Stack

Layer	Technology
Test Framework	Playwright (TypeScript / JavaScript / Python)
Integration Scripts	Groovy (`protonIntegration.groovy`, `validateProtonTestCase.groovy`)
CI/CD	Jenkins (`Jenkinsfile` — all three language runners updated)
Test Management	Proton (via REST API, JSON/HTTPS)
Reporting	`results.json` + PDF (existing format, reused as-is)
ID Mapping	Jira IDs (e.g., `DNIGMP-188`) mapped to Proton test case IDs

Code Reuse

Approximately **80% of the existing Xray integration code** will be reused, with the Proton integration module replacing only the API client and endpoint-specific logic.

---

4. Effort Estimate

Activity Breakdown

Activity	Effort (Person Days)	Notes
Discovery, Design & Development	40	API analysis, integration design, implementation, pipeline updates
Testing and Validation	20	Sandbox testing, regression, edge cases, CI/CD validation
Rework as per UAT Feedback	10	Incorporating stakeholder feedback post-UAT
Total Person Days	**70**	
Total Person Months	**3**	Based on 21 working days/month

Key Metrics

Parameter	Value
FTE Required	1
Working Days per Month	21
Project Duration	3 Months

---

5. Project Timeline

13-Week Delivery Plan

Weeks	Phase	Key Deliverables
1–3	Discovery	Proton API documentation reviewed, connectivity confirmed, concept map produced, design document signed off by stakeholders
4–7	Core Integration	`protonIntegration.groovy` — full implementation covering Standalone execution, Regression execution, and error handling
8–9	Approval Validation	`validateProtonTestCase.groovy` — approval status check wired into pipeline pre-execution checks (mirrors existing Xray pattern)
9	Pipeline Wiring	All three language runners (TypeScript / JavaScript / Python) and `Jenkinsfile` updated to route through Proton integration
10–12	Testing & Validation	End-to-end tests across all language runners, failure scenario coverage, sandbox regression runs
12–13	UAT & Rework	UAT execution with Proton stakeholders, rework based on feedback, production deployment, knowledge transfer

Milestone Summary

Milestone	Target Week
Design Document Signed Off	Week 3
`protonIntegration.groovy` Complete	Week 7
`validateProtonTestCase.groovy` & Pipeline Wiring Complete	Week 9
End-to-End Testing Complete (all runners)	Week 12
UAT Sign-Off & Production Go-Live	Week 13

---

6. Team Structure

Proposed Team

Role	Headcount	Responsibilities
Senior Test Automation Engineer	1 (FTE)	End-to-end delivery: API analysis, design, development, testing, UAT support, documentation

Collaboration Requirements (Proton Team)

The Proton team is requested to provide approximately **3 days of collaboration** spread across the 12-week engagement, covering:

- API onboarding and credential/access provisioning
- Clarification of test case ID mapping and data model queries
- UAT participation and sign-off

---

7. Commercials

Pricing Summary

Parameter	Value
Engagement Model	Fixed Price
Average Day Rate (ADR)	CHF 181
Working Days per Month	21
Monthly Cost	CHF 3,801
Project Duration	3 Months
Total Project Cost	**CHF 11,403**

Cost Breakdown by Phase

Phase	Person Days	Cost (CHF)

Discovery, Design & Development	40	7,240
Testing and Validation	20	3,620
Rework as per UAT Feedback	10	1,810
Total	**70**	**12,670**

> ***Note:*** *The fixed project cost is CHF **11,403** based on the agreed monthly rate. Any scope changes beyond what is defined in Section 2 will be subject to a formal change request and revised pricing.*

Payment Terms

Milestone	Payment (%)	Amount (CHF)
Project Kickoff	30%	3,421
Development Complete (Week 9)	40%	4,561
UAT Sign-Off & Go-Live (Week 12)	30%	3,421
Total	**100%**	**11,403**

---

8. Assumptions and Dependencies

Assumptions

#	Assumption
A1	Proton exposes a **REST API (JSON/HTTPS)** that supports creating test runs, submitting results, and uploading file attachments programmatically. Without this, integration is not feasible.
A2	Proton test cases use the **same IDs as Jira** (e.g., `DNIGMP-188`) or a clear, queryable mapping exists between the two systems.
A3	Proton exposes a **test case approval status via API** so the pipeline can validate approval before execution — same pattern as the current Xray check.
A4	A **Proton sandbox/test project** is available for development and testing (Weeks 4–12) to avoid polluting live data.
A5	The **existing PDF report format** and `results.json` structure are accepted by Proton stakeholders without modification (~80% of existing code is reused as-is).
A6	The **Proton team is available for approximately 3 days** of collaboration (API questions, test data, UAT sign-off) spread across the 12 weeks.
A7	Existing CI/CD pipeline infrastructure is in place and accessible; no new infrastructure provisioning is required as part of this engagement.
A8	All required API credentials, access tokens, and sandbox environment details will be provided to the Altimetrik team by the end of Week 1.

Dependencies

#	Dependency	Owner	Required By
D1	Proton REST API documentation and access credentials	Proton Team	Week 1
D2	Proton sandbox environment provisioned and accessible	Proton Team	Week 4
D3	Confirmation of Jira-to-Proton ID mapping approach	Proton Team	Week 3
D4	Approval status API endpoint confirmed and documented	Proton Team	Week 3
D5	UAT participants identified and availability confirmed	Customer	Week 10
D6	Access to existing Playwright framework codebase and CI/CD pipeline	Customer / Altimetrik	Week 1

Risks

#	Risk	Likelihood	Impact	Mitigation
R1	Proton API does not support required operations (test run creation, attachment upload)	Low	High	Early API discovery in Week 1–2; escalate immediately if gaps identified
R2	Jira-to-Proton ID mapping is complex or requires custom logic	Medium	Medium	Allocate buffer in design phase; clarify mapping by Week 3
R3	Proton sandbox unavailable during development	Low	High	Confirm sandbox access before Week 4; raise as blocker if delayed
R4	UAT feedback requires significant rework beyond 10 days	Low	Medium	Early stakeholder alignment on acceptance criteria before UAT begins

---

Acceptance

Role	Name	Signature	Date
Prepared by (Altimetrik)			
Approved by (Customer)			

This proposal is valid for 30 days from the date of issue.
All costs are in CHF and exclusive of applicable taxes unless otherwise stated.